DCC Summary

Lexi Aiello

October 7, 2026

Abstract

• The dependent combinator calculus (DCC) is a dependent type theory based on categorical combinators [**CURIEN1986188**] and affine logic.

- DCC is similar to the base calculus of constructions in that types are terms and that there is an infinite sort hierarchy (but no universe-level polymorphism).
- Propositional equality is inductively generated from $refl$ as in intuitionistic type theory.
- DCC has decidable type-checking, global confluence, and strong normalization.
- A subset of Lean 4 can be compiled to DCC.
- β-reduction always terminates in DCC due to affine logic. We eagerly β and η-reduce all types in each constructor, rather than assuming a general conversion axiom. This enables extensional equality reflection, but the proof has a *sorry* for subject-reduction.
- η-reduction is based on observing the behavior of a function on typed "holes" which are generated from the domain of the function, then reducing to the "canonical" combinator producing the hole output (similarly to in NbE). Since we do not assume inductive types and since we do not have recursors on 2, we believe we may have computable function extensionality, but this fact is hard to show formally.

1 Base Types

Our base types are the usual as in MLTT.

Definition 1. *We have an infinite sort hierarchy:*

$$\frac{}{Sort_u : Sort_{u+1}} \tag{1}$$

Definition 2. *We have a bottom type:*

$$\frac{}{\mathbb{0} : Sort_0} \tag{2}$$

Definition 3. *We have a top type with one inhabitant:*

$$\frac{}{\mathbb{1} : Sort_0} \tag{3}$$

$$\frac{}{* : \mathbb{1}} \tag{4}$$

Definition 4. *We have a type 2:*

$$\frac{}{2 : Sort_1} \tag{5}$$

It has two inhabitants:

$$\frac{}{\mathbb{0}_2 : 2} \tag{6}$$

$$\frac{}{\mathbb{1}_2 : 2} \tag{7}$$

2 Syntax

The syntax is given in S-expressions for clarity.

Definition 5. *An expression is either:*

- *A base term inhabiting a base type*
- *A base type*
- *A parenthesized expression beginning with a combinator C and containing its points $p_1 \dots p_n$ separated by whitespace: $(C\ p_1 \dots\ p_n)$.*
- *A juxtaposed list of expressions separated by whitespace, denoting an application.*
- *A parenthesized expression*

Example 1. *$(Id\ A)$ is an expression.*

Proof. See 5. □

Example 2. *$((Id\ A)x)$ is an expression.*

Proof. See 5. □

3 Core Combinators

DCC is an extension of the simply-typed categorical combinators [**CURIEN1986188**] to dependent types and affine logic. Our choice of the categorical combinators is due to their close correspondence with cartesian closed categories. We support affine logic by "box" and "bang" terms:

Definition 6. *A bang type $(!A)$ is inhabited by a boxed term $(\&x) : (!A)$:*

$$\textsc{Box}\ \frac{x : A}{(\&x) : (!A)} \tag{8}$$

$$\textsc{Bang}\ \frac{A : Sort_u}{(!A) : Sort_u} \tag{9}$$

Combinators' type arguments are given as points of the combinators, but we assume the existence of Σ-types, so expressivity as if type arguments were given by application is recovered since Σ-pairs can encode telescope contexts.

Definition 7. *The head-reduction rules on combinators are as follows:*

$$\textsc{Id}\ \frac{}{(Id\ A)x \to_{\beta_{wh}} x} \qquad \textsc{Fst}\ \frac{}{(Fst\ B)(,B_2\ x\ y) \to_{\beta_{wh}} x} \qquad \textsc{Snd}\ \frac{}{(Snd\ B)(,B_2\ x\ y) \to_{\beta_{wh}} y}$$

$$\textsc{Curr}\ \frac{}{(\Lambda\ A\ B\ x)yz \to_{\beta_{wh}} (x(,B\ y\ z))} \qquad \textsc{Comp}\ \frac{}{(\circ\ x\ y)z \to_{\beta_{wh}} (x(yz))} \qquad \textsc{Quot}\ \frac{}{(`B\ x)y \to_{\beta_{wh}} x}$$

$$\textsc{Curr'}\ \frac{}{(!\Lambda\ A\ B\ x)(\&\ x)(\&\ y) \to_{\beta_{wh}} (x(\&\ (,B\ y\ z)))} \qquad \textsc{Pair}\ \frac{}{(\langle,\rangle B\ x\ y)(\&\ x) \to_{\beta_{wh}} (,(Bx)\ (Mx)\ (Nx(Mx)))}$$

$$\textsc{Eval}\ \frac{}{(Eval\ A\ B\ x)(,B_2\ y\ z) \to_{\beta_{wh}} xyz}$$

$$\textsc{Unbox}\ \frac{}{(*\ A)(\&\ x) \to_{\beta_{wh}} x} \qquad \textsc{Bang map}\ \frac{}{(!<>\ f)(\&x) \to_{\beta_{wh}} (!(fx))} \qquad \textsc{Box map}\ \frac{}{(\$<>\ f)(\&x) \to_{\beta_{wh}} (\&(fx))}$$

$$\textsc{Eql}\ \frac{}{(Eql\ B)x(,B_2\ y\ z) \to_{\beta_{wh}} (=\ (Bx)\ y\ z)} \qquad \textsc{Sigma}\ \frac{}{(\Sigma\ B\ C)x \to_{\beta_{wh}} (\Sigma(Cx))}$$

There are also some basic operations on un-boxed Σ-pairs added as optimizations to avoid boxing the term and using the pair rule:

$$\textsc{Dup}\quad \frac{}{(Dup\ B\ x)(\&y)(\&z) \to_{\beta_{wh}} (xy(,(`(By)\ (By))\ z\ z))}$$

$$\textsc{Bimap}\quad \frac{}{(Map\ A\ B\ f\ g)(,B_2\ x\ y) \to_{\beta_{wh}} (,(`A\ B)\ (fx)\ (gy))}$$

$$\textsc{Assoc}\quad \frac{}{(Assoc\ A\ B\ C)(,B_2\ (,B_3\ x\ y)\ z) \to_{\beta_{wh}} (,(`A\ (\Sigma(`BC)))\ x\ (,(`B\ C)\ y\ z))}$$

DCC uses call-by-value semantics.

Definition 8. *The full β-reduction rules are as such:*

$$\textsc{Congr Fun.}\quad \frac{f \to_\beta f'}{(fx) \to_\beta (f'x)} \qquad \textsc{Congr App.}\quad \frac{x \to_\beta x'}{(fx) \to_\beta (fx')} \qquad \textsc{Head}\quad \frac{(fx) \to_{\beta_{wh}} B}{(fx) \to_\beta B}$$

$$\textsc{Congr Point}\quad \frac{points(f,n) \to_\beta p'}{f \to_\beta points.set(f,n,p')}$$

Example 3. $(,B\ ((Id\ A)x)\ y) \to_\beta (,B\ x\ y)$

Proof. By the congr. point rule since is a combinator with point arity 3, and since $(Id\ A)x \to_{\beta_{wh}} x$. □

3.1 Strong Normalization by Syntactic Size Measure

DCC is provably strongly normalizing by a syntactic size measure on expressions, since the only rule that duplicates a term is the pair rule (as in categorical combinators). We simply count the size of a boxed term thrice so that pair decreases the size of the term $size(\&x) \coloneqq size(x) * 3 + 1$. The size measure is relatively obtuse and arbitrary, but strong normalization is a trivial result of affine logic, so the details have been omitted. We separately show that DCC is locally confluent, and thus inherit global confluence by Newmann's Lemma.

4 η-reduction

DCC is a minimal calculus, and we do not have inductives or recursors for 𝟚. So, a function f is η-reduced by characterizing its behavior on a "hole" matching the shape of their domain. A hole is either an opaque term indexed by a unique level ($hole(n)$), a Σ-pair containing two holes, or a boxed hole. The η-reduct of f is defined to be the canonical combinator yielding the β-normal form of $fhole(f)$ with its domain substituted back to the domain of f.

Example 4. $(\circ(Fst\ B)(Id\ (\Sigma B))) \to_\eta (Fst\ B)$

Proof. This is since $hole((\circ(Fst\ B)(Id\ (\Sigma B)))) \coloneqq (hole(1), hole(Dom(B)))$ and $(\circ(Fst\ B)(Id\ (\Sigma B))) \to_\beta hole(1)$. So, the η-reduct is defined to be $Fst\ B$. □

We have omitted most of the details for brevity.

5 Inference Rules

DCC is a pure calculus, so there is no distinction between arrows and Π-types, nor between Σ and products. Since DCC is strongly normalizing, we eagerly reduce $(\eta \circ \beta)$ each type in the consequent of each inference rule, rather than allowing a catch-all conversion rule. This theoretically permits equality reflection, since $h : (=\ B\ x\ y)$ iff $x \equiv y$, but we have a "sorry" in the proof for subject-reduction.

Definition 9. *The inference rule for* Π *is as such:*

$$\frac{B : (env : A) \to Sort_u \qquad C : (env : A) \to (x : Benv) \to Sort_v}{(\Pi BC) : (env : A) \to Sort_{\max(u,v)}} \tag{10}$$